



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий
(МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Тестирование и верификация программного обеспечения»

Практическое занятие № 2

*ИКБО-43-23 ОГАНЕСЯН Д. К.
КОЩЕЕВ М.И.
ГУТКОВИЧ А.А.
ИВАНОВ А.В.
УЛЗЕТУЕВ А.С.*

(подпись)

АССИСТЕНТ

ИЛЬИЧЕВ Г.П.

(подпись)

**ОТЧЕТ
ПРЕДСТАВЛЕН**

«17»__ОКТЯБРЯ_2025 Г.

Москва 2025 г.

СОДЕРЖАНИЕ

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ.....	3
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	4
2.1 Разработка модулей.....	4
2.2 Модульное тестирование.....	5
2.3 Мутационное тестирование.....	7
3 АНАЛИЗ И ВЫВОДЫ.....	12
3.1 Оценка качества тестирования.....	12
3.2 Анализ недостатков и их устранение.....	12
3.3 Итоговые выводы по результатам работы.....	13

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

Цель работы: познакомить студентов с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Для достижения поставленной цели работы студентам необходимо выполнить ряд задач:

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Разработка модулей

Описание функциональности: Модуль содержит набор функций для валидации входных данных при конвертации: проверяет, является ли введенное значение корректным числом; убеждается, что число положительное; контролирует, находится ли температура в допустимом диапазоне для выбранной шкалы (Цельсия, Фаренгейта или Кельвина); проверяет существование указанной единицы измерения в заданной категории; а также осуществляет проверку наличия самой категории в системе единиц измерения, при этом в случае обнаружения ошибок генерирует соответствующие сообщения об ошибках.

Листинг 4 — Исходный код.

```
# Функция проверки является ли числом
def validate_numeric_input(value_str):
    try:
        return float(value_str)
    except ValueError:
        raise ValueError("Ошибка: введите корректное число")

# Функция проверки что число положительное
def validate_positive_number(value):
    if value <= 0:
        raise ValueError("Ошибка: значение должно быть положительным")
    return value

# Функция проверки допустимости температуры
def validate_temperature_range(value, scale):
    ranges = {
        'c': (-273.15, 10000),
        'f': (-459.67, 18032),
        'k': (0, 10273.15)
    }
    min_temp, max_temp = ranges.get(scale, (-float('inf'), float('inf')))
    if not min_temp <= value <= max_temp:
        raise ValueError(f"Температура должна быть в диапазоне {min_temp} - {max_temp}")
    return value

# Функция проверки существования единицы измерения
def validate_unit_exists(category, unit):
    from core.convert_script import UNITS
    if category not in UNITS:
        raise ValueError(f"Категория '{category}' не найдена")
    if unit not in UNITS[category]:
        raise ValueError(f"Единица '{unit}' не найдена в категории '{category}'")
    return True
```

```
# Функция проверки существования категории
def validate_category(category):
    from core.convert_script import UNITS
    if category not in UNITS:
        raise ValueError(f"Категория '{category}' не найдена")
    return True
```

2.2 Модульное тестирование

Описание тестов: Тестовый набор направлен на проверку корректности работы модуля валидации входных данных, включая проверку числовых значений, их положительности, температурных диапазонов, существования единиц измерения и категорий конвертации.

Листинг 12 — Код тестов.

```
import pytest
import sys
import os
import core.validator as val

def test_validate_numeric_input_valid():
    assert val.validate_numeric_input("42") == 42.0
    assert val.validate_numeric_input("3.14") == 3.14
    assert val.validate_numeric_input("-10") == -10.0

def test_validate_numeric_input_invalid():
    with pytest.raises(ValueError, match="Ошибка: введите корректное число"):
        val.validate_numeric_input("abc")

def test_validate_positive_number_valid():
    assert val.validate_positive_number(5) == 5
    assert val.validate_positive_number(0) == 0
    assert val.validate_positive_number(0.001) == 0.001

def test_validate_positive_number_invalid():
    with pytest.raises(ValueError, match="Ошибка: значение должно быть положительным"):
        val.validate_positive_number(-5)

def test_validate_temperature_range_valid():
    assert val.validate_temperature_range(0, 'c') == 0
    assert val.validate_temperature_range(-273.15, 'c') == -273.15
    assert val.validate_temperature_range(32, 'f') == 32
    assert val.validate_temperature_range(273.15, 'k') == 273.15

def test_validate_temperature_range_invalid():
    with pytest.raises(ValueError):
        val.validate_temperature_range(-274, 'c')
    with pytest.raises(ValueError):
        val.validate_temperature_range(-1, 'k')

def test_validate_unit_exists_valid():
    assert val.validate_unit_exists("длина", "метр") == True
    assert val.validate_unit_exists("масса", "грамм") == True
```

```

def test_validate_unit_exists_invalid():
    with pytest.raises(ValueError):
        val.validate_unit_exists("длина", "фут")
    with pytest.raises(ValueError):
        val.validate_unit_exists("объем", "литр")

def test_validate_category_valid():
    assert val.validate_category("длина") == True
    assert val.validate_category("масса") == True

def test_validate_category_invalid():
    with pytest.raises(ValueError):
        val.validate_category("объем")

```

Методология: Тестирование осуществляется через комбинацию проверок корректных и некорректных входных данных с использованием механизма перехвата исключений. Особое внимание уделяется валидации граничных значений и обработке ошибок при работе с различными единицами измерений.

Анализ покрытия кода: Тесты охватывают все основные функции валидации: проверку числовых значений, контроль положительности чисел, валидацию температурных диапазонов для разных шкал, проверку существования единиц измерения и категорий конвертации. Проверяется как корректная работа с допустимыми значениями, так и обработка ошибочных ситуаций.

Исправление ошибок:

Краткое описание ошибки: «Неверная проверка положительного числа».

Статус ошибки: открыта («Open»).

Категория ошибки: серьезная («Major»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Загрузить тест.

```
result = core.validator.validate_positive_number(0)
```

2. Полученный результат: «ValueError("Ошибка: значение должно быть положительным")». Ожидаемый результат: «0».

Листинг 13 — Исправленный код.

```
# Функция проверки является ли числом
def validate_numeric_input(value_str):
    try:
        return float(value_str)
    except ValueError:
        raise ValueError("Ошибка: введите корректное число")

# Функция проверки что число положительное
def validate_positive_number(value):
    if value < 0:
        raise ValueError("Ошибка: значение должно быть положительным")
    return value

# Функция проверки допустимости температуры
def validate_temperature_range(value, scale):
    ranges = {
        'c': (-273.15, 10000),
        'f': (-459.67, 18032),
        'k': (0, 10273.15)
    }
    min_temp, max_temp = ranges.get(scale, (-float('inf'), float('inf')))
    if not min_temp <= value <= max_temp:
        raise ValueError(f"Температура должна быть в диапазоне {min_temp} - {max_temp}")
    return value

# Функция проверки существования единицы измерения
def validate_unit_exists(category, unit):
    from core.convert_script import UNITS
    if category not in UNITS:
        raise ValueError(f"Категория '{category}' не найдена")
    if unit not in UNITS[category]:
        raise ValueError(f"Единица '{unit}' не найдена в категории '{category}'")
    return True

# Функция проверки существования категории
def validate_category(category):
    from core.convert_script import UNITS
    if category not in UNITS:
        raise ValueError(f"Категория '{category}' не найдена")
    return True
```

2.3 Мутационное тестирование

Создание мутантов:

1. Мутации в validate_numeric_input

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
try: return float(value_str)	try: return int(value	test_validate_numeric_input_valid	 Да	Нет

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
	<code>_str)</code> (Изменение функции конвертации)	(для "3.14") Провалится (ValueError или 3 вместо 3.14)		
<code>raise ValueError(...)</code>	Удаление строки (Удаление оператора)	<code>test_validate_numeric_input_invalid</code> Провалится (тест ожидает исключение, но его нет)	 Да	Нет

2. Мутации в `validate_positive_number`

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
<code>if value < 0:</code>	<code>if value <= 0:</code> (Изменение оператора)	<code>test_validate_positive_number_valid</code> (для 0) Провалится (тест ожидает 0, но получит ValueError)	 Да	Нет
<code>raise ValueError(...)</code>	Удаление строки (Удаление оператора)	<code>test_validate_positive_number_invalid</code> Провалится	 Да	Нет

3. Мутации в `validate_temperature_range`

Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
<code>ranges.get(scale, (-float('inf'</code>	<code>ranges.get(scale, (0, 0))</code>	Выживет	 Нет	Да

Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
<code>), float('inf'))</code>	(Изменение значения по умолчанию)			
<code>min_temp <= value <= max_temp</code>	<code>min_temp < value <= max_temp</code> (Изменение оператора)	<code>test_validate_temperature_range_valid</code> (для \$-273.15\$ и \$0\$K) Провалится	 Да	Нет
<code>min_temp <= value <= max_temp</code>	<code>min_temp <= value</code> (Удаление части сравнения)	Выживет	 Нет	Да

4. Мутации в `validate_unit_exists` и `validate_category`

Функция	Исходный код	Мутант (Ошибка)	Результат теста	Убит ?	Недостаток теста?
<code>validate_unit_exists</code>	<code>if category in UNITS:</code> (Инверсия условия)	<code>test_validate_unit_exists_valid</code> Провалится (выдаст ошибку, когда не должен)	 Да	Нет	
<code>validate_unit_exists</code>	<code>if unit in UNITS[category]:</code> (Инверсия условия)	<code>test_validate_unit_exists_invalid</code> (для "фут") Провалится (выдаст ошибку <code>KeyError</code> или	 Да	Нет	

Функция	Исходный код	Мутант (Ошибка)	Результат теста	Убит?	Недостаток теста?
		ValueError раньше времени)			
validate_category	if category in UNITS: (Инверсия условия)	test_validate_category_invalid Провалится	 Да	Нет	
validate_unit_exists	return False (Изменение возвращаемого значения)	test_validate_unit_exists_valid Провалится (False != True)	 Да	Нет	

Анализ их выживания:

Мутант 1 (Диапазон по умолчанию): Наш тест не включал случай неизвестной шкалы (например, rankine). Если изменить значение по умолчанию, тесты выживут. Мы добавили явную проверку, что неизвестная шкала не вызывает исключений.

Мутант 2 (Удаление верхней границы): Удаление проверки `value <= max_temp` позволило мутанту выжить, так как наши тесты не включали проверку верхних границ (например, 10001°C).

Корректировка тестов:

1. Чтобы убить выжившего мутанта (Верхние границы температуры), мы добавили тесты, которые проверяют, что максимальные значения действительно являются границей, и что числа, превышающие их, вызывают ошибку.

Листинг 23 — Код теста.

```
def test_validate_temperature_range_upper_limit():
    # Проверка на максимальное значение (должно работать)
```

```
assert val.validate_temperature_range(10000, 'c') == 10000

# Проверка на значение выше максимума (должно провалиться)
with pytest.raises(ValueError):
    val.validate_temperature_range(10000.0001, 'c') # Убивает мутанта с
удалением верхней границы

# Добавьте аналогичный тест для шкалы Фаренгейта (18032)
with pytest.raises(ValueError):
    val.validate_temperature_range(18033, 'f')
```

2. Чтобы убить выжившего мутанта (Неизвестная шкала температуры), мы добавили тест, который проверяет поведение функции при передаче неизвестной шкалы.

Листинг 24 — Код теста.

```
def test_validate_temperature_range_unknown_scale():
    # Неизвестная шкала должна иметь бесконечный диапазон (-inf, inf)
    assert val.validate_temperature_range(1000000, 'rankine') == 1000000
    assert val.validate_temperature_range(-5000000, 'rankine') == -5000000 #
Убивает мутанта с конечным диапазоном по умолчанию
```

3 АНАЛИЗ И ВЫВОДЫ

3.1 Оценка качества тестирования

Итоговое Качество Тестирования:

1. Полнота (Покрытие): Теперь тесты обеспечивают не только высокое покрытие строк кода, но и высокое покрытие логики. Почти все значимые операторы (математические, логические, сравнения) проверяются на их чувствительность к ошибкам.
2. Надежность (Чувствительность): Набор тестов способен обнаруживать даже атомарные, синтаксически небольшие ошибки (мутанты), которые могли бы быть допущены программистом, что является прямым доказательством его надежности.
3. Тестирование интеграционных точек: Особое внимание уделено интеграционным точкам (например, view вызывает validator и history). Проверка, что удаление валидации приводит к сбою, а удаление записи в историю обнаруживается, делает интеграционные тесты более надежными.

3.2 Анализ недостатков и их устранение

Улучшения, которые были достигнуты благодаря мутационному тестированию:

Модуль	Ранее Выжившие Мутанты (Пробелы)	Статус после изменений
validator	Недостаточно проверялись верхние границы температурных диапазонов и обработка неизвестных шкал.	Убиты. Добавлены тесты для верхних границ и для неизвестных шкал (rankine), что гарантирует корректную работу логики по умолчанию (-inf, +inf).

3.3 Итоговые выводы по результатам работы

В результате выполнения практической работы были достигнуты поставленные цели: освоены методы модульного и мутационного тестирования, разработаны и проанализированы тестовые наборы. В ходе исследования успешно применено мутационное тестирование, созданы эффективные модульные тесты с высоким покрытием кода. Мутационное тестирование позволило выявить и устранить пробелы в тестовом наборе, повысив качество проверки программного продукта. Приобретены практические навыки разработки тестов, анализа покрытия кода и оценки эффективности тестовых сценариев. Полученные результаты подтверждают, что комбинированный подход к тестированию обеспечивает надёжную проверку качества программного обеспечения и помогает обнаруживать потенциальные ошибки на ранних этапах разработки.